

SEO-Focused Static Site Generator in Rust

Overview

This project is a high-performance, SEO-focused Static Site Generator (SSG) written in Rust, targeting developers, bloggers, and businesses who want to generate static websites quickly and with full SEO optimization.

Project name ideas: *FlashSite*, *RustPress*, *BlitzGen*

Core Features

- Markdown to HTML rendering
- Templating using Tera
- Static asset bundling (CSS/JS/images)
- File watcher for hot rebuilds
- Deployment support to CDNs (e.g., Netlify, GitHub Pages, Cloudflare)

SEO-Focused Features

- Auto-generation of `sitemap.xml`, `robots.txt`, and canonical URLs
- Open Graph and Twitter Card metadata injection
- Clean URLs with automatic redirect support
- Structured data generation (JSON-LD for articles, products, etc.)
- Keyword density and readability checker
- Image `alt` tag validator

Project Setup

1. Create a new Rust project

```
cargo new rust_seo_ssg --bin
cd rust_seo_ssg
```

2. Dependencies (Cargo.toml)

```
[dependencies]
pulldown-cmark = "0.10"
tera = "1.18"
walkdir = "2.4"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
chrono = "0.4"
sitemap = "0.5"
```

3. Basic Build Workflow

1. Read Markdown files from `content/`
2. Convert to HTML using `pulldown-cmark`
3. Apply templates using `Tera`
4. Output static HTML into `public/`
5. Generate SEO artifacts: `sitemap.xml`, `robots.txt`

4. Sitemap Generator

```
use sitemap::writer::SiteMapWriter;
use std::fs::File;

fn generate_sitemap(urls: Vec<String>) {
    let file = File::create("public/sitemap.xml").unwrap();
    let mut sitemap = SiteMapWriter::new(file);
    let mut urlwriter = sitemap.start_urlset().unwrap();

    for url in urls {
        urlwriter.url(url.as_str()).unwrap().build().unwrap();
    }

    urlwriter.end().unwrap();
}
```

5. robots.txt Generator

```
use std::fs;

fn generate_robots_txt() {
    let robots = "User-agent: *\nAllow: /\nSitemap: https://\nyoursite.com/sitemap.xml";
    fs::write("public/robots.txt", robots).unwrap();
}
```

Deployment Feature

Add CLI options like `build`, `serve`, and `deploy`. Deployment could target:

- GitHub Pages
- Netlify via API
- Cloudflare Pages

Bonus Features (for Premium Version)

- GUI frontend using Tauri
- Blogging mode with tags, RSS, and pagination
- Integrated link checker and image optimizer
- Basic SEO audit with keyword analysis

Monetization Ideas

- Open core + paid Pro version with GUI
- Subscription-based one-click deployment (SaaS)
- Theme marketplace
- Paid SEO auditing add-on

Next Steps

1. Build Markdown to HTML + templating engine
2. Add routing and sitemap generation
3. Integrate deployment and SEO tools
4. Publish open-source core on GitHub
5. Plan Pro/premium feature set